

CS 437 Lab 3: Steps 1–2 Security Report

Date: October 27, 2025

1 Overview

This document consolidates evidence and reflections for the firewalling and packet-inspection portions of Lab 3. Automation scripts `part1_firewall_setup.sh` and `part2_packet_inspection.sh` were executed on 27 Oct 2025, producing reproducible logs inside `part1_firewall_outputs/` and `part2_packet_outputs/`. Figures in `figures/` capture required snapshots for the grading rubric.

2 Step 1: Firewalling

2.1 Background Reading

- [Firewall \(computing\) – Wikipedia](#) and cross-links on packet filtering, stateful inspection, and application gateways.
- Raspberry Pi Documentation – Configuration (security best practices) alongside DigitalOcean’s “UFW Essentials” article for rule syntax patterns.

2.2 Key Takeaways

IoT-appropriate firewall types. Stateful packet inspection offers a practical middle ground for Pi-class hardware: it preserves connection awareness without the overhead of deep packet inspection. Layer-7 filtering tailored to MQTT, HTTPS, and SSH strengthens anomaly detection by constraining traffic semantics, while perimeter next-generation firewalls can shoulder heavier inspection for the broader deployment.

Defense-in-depth layers. I combined perimeter filtering on the lab Wi-Fi, UFW on the Pi, and application-level authentication on the teleoperation API. The layered approach ensures that misconfigurations in any single tier do not immediately expose the robot, and it limits blast radius if credentials leak.

Risks and mitigations. Common IoT threats include botnet recruitment, credential stuffing, and lateral movement post-compromise. Tight UFW rules restrict exposed services, reducing opportunities for automated scans, and numbered rule logging provides early indicators of probing that complement intrusion-detection tooling.

2.3 Firewall Configuration Evidence

Running `part1_firewall_setup.sh` captured the transcript in `part1_firewall_outputs/ufw_setup_20251027-121647.log`. An excerpt highlighting the rule application and verification appears below.

Listing 1: Excerpt from UFW configuration log

```
$ sed -n '70,120p' part1_firewall_outputs/ufw_setup_20251027-121647.log
===== Apply allow rules =====
===== Allow ssh =====
+ sudo ufw allow ssh
```

```

Rules updated
Rules updated (v6)

===== Allow 1883/tcp =====
+ sudo ufw allow 1883/tcp
Rules updated
Rules updated (v6)

===== Allow 8883/tcp =====
+ sudo ufw allow 8883/tcp
Rules updated
Rules updated (v6)

===== Allow 5000/tcp =====
+ sudo ufw allow 5000/tcp
Rules updated
Rules updated (v6)

===== UFW status =====
+ sudo ufw status numbered
Status: active

```

	To	Action	From
	--	-----	----
[1]	22/tcp	ALLOW IN	Anywhere
[2]	1883/tcp	ALLOW IN	Anywhere
[3]	8883/tcp	ALLOW IN	Anywhere
[4]	5000/tcp	ALLOW IN	Anywhere
[5]	22/tcp (v6)	ALLOW IN	Anywhere (v6)
[6]	1883/tcp (v6)	ALLOW IN	Anywhere (v6)
[7]	8883/tcp (v6)	ALLOW IN	Anywhere (v6)
[8]	5000/tcp (v6)	ALLOW IN	Anywhere (v6)

2.4 Allowed Services Summary

- Default policy: deny incoming, allow outgoing (set via `ufw default`).
- Allowed sharable ports: `ssh`, MQTT broker on `1883/tcp`, MQTT over TLS on `8883/tcp`, and the Lab 2 teleoperation service on `5000/tcp`. No other listeners remain exposed.
- Evidence bundles: `summary_20251027-121647.txt` and `allowed_rules_20251027-121647.txt` inside `part1_firewall_outputs/` document the rule set at activation time.

3 Step 2: Packet Inspection

3.1 Background Reading

- [pcap – Wikipedia](#) and [Comparitech’s PCAP guide](#) for capture-format fundamentals.
- [Wireshark – Wikipedia](#) (“Features” section) and [“Wireshark Tutorial for Beginners”](#) video for feature walkthroughs.

Status: active

	To	Action	From
	--	-----	----
[1]	22/tcp	ALLOW IN	Anywhere
[2]	1883/tcp	ALLOW IN	Anywhere
[3]	8883/tcp	ALLOW IN	Anywhere
[4]	5000/tcp	ALLOW IN	Anywhere
[5]	22/tcp (v6)	ALLOW IN	Anywhere (v6)
[6]	1883/tcp (v6)	ALLOW IN	Anywhere (v6)
[7]	8883/tcp (v6)	ALLOW IN	Anywhere (v6)
[8]	5000/tcp (v6)	ALLOW IN	Anywhere (v6)

Figure 1: Numbered UFW rules after applying Part 1 policies.

3.2 Key Takeaways

pcap capabilities and limits. pcap preserves timestamped packet headers and payload bytes, enabling replay of TCP handshakes, inspection of DNS queries, and latency analysis. It excels at diagnosing retransmissions, misrouted traffic, and protocol misuse. However, encrypted payloads remain opaque, application semantics beyond the transport layer require additional tooling, and very large captures can overwhelm constrained storage.

Wireshark in context. Wireshark is an interactive analyzer that ingests pcap streams (live or offline) and layers on dissection, filtering, reassembly, and visualization. Whereas pcap is a file format, Wireshark (and its CLI sibling `tshark`) is the toolchain that interprets those bytes. Typical uses include validating TLS handshakes, tracing MQTT publishes, isolating packet loss, and spotting unexpected broadcast traffic.

3.3 Capture Setup and Evidence

The helper script `part2_packet_inspection.sh` installed Wireshark/Tshark, captured 20 seconds of traffic on `wlan0`, and saved artefacts in `part2_packet_outputs/`. The transcript `packet_inspection_20251027-` shows the interface enumeration, latency probes, and capture metadata.

Listing 2: Excerpt from packet inspection log

```
$ sed -n '60,150p' part2_packet_outputs/packet_inspection_20251027-123450.log
===== Packet capture session =====
Capturing on interface wlan0 for 20 seconds
PCAP destination: /home/desi/CS437/Lab3/part2_packet_outputs/capture_20251027-123450.
    pcapng
tshark PID 9219
===== Ping 8.8.8.8 =====
+ ping -c 6 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=119 time=8.28 ms
...
```

```

===== HTTP HEAD https://example.com =====
+ curl -fsSI --max-time 10 https://example.com
HTTP/2 200
content-type: text/html
etag: "bc2473a18e003bdb249eba5ce893033f:1760028122.592274"
last-modified: Thu, 09 Oct 2025 16:42:02 GMT
cache-control: max-age=86000
...
===== capinfos summary =====
+ capinfos /home/desi/CS437/Lab3/part2_packet_outputs/capture_20251027-123450.pcapng
File type:           Wireshark/... - pcapng
Number of packets:   404
Capture duration:    18.276217448 seconds
Average packet rate: 22 packets/s

```

```

Capture: /home/desi/CS437/Lab3/part2_packet_outputs/capture_20251027-123450.pcapng
Interface: wlan0
Ping target: 8.8.8.8
HTTP URL: https://example.com

```

Protocol hierarchy (tshark -z io,phs):

```

=====
Protocol Hierarchy Statistics
Filter:

```

```

eth                      frames:404 bytes:118718
  ip                      frames:397 bytes:118123
    tcp                  frames:368 bytes:114200
      tls                frames:81 bytes:48539
        tcp.segments    frames:7 bytes:6560
          tls            frames:1 bytes:1362
            tcp.segments frames:4 bytes:6032
              ssh        frames:99 bytes:19710
                icmp      frames:12 bytes:1176
                  udp      frames:17 bytes:2747
                    dns     frames:16 bytes:2532
                      mndp   frames:1 bytes:215
                        arp    frames:6 bytes:360
                          ipv6 frames:1 bytes:235
                            udp  frames:1 bytes:235
                              mndp frames:1 bytes:235

```

Endpoint summary (tshark -z endpoints,ip):

```

=====
IPv4 Endpoints
Filter:<No Filter>

```

	Packets	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes
192.168.125.22	396	117908	193	57598	203	60310

Figure 2: Endpoint and protocol hierarchy summary generated from the captured pcap.

Evidence bundle highlights:

- Capture artefact: `capture_20251027-123450.pcapng`.
- Analysis digests: `analysis_20251027-123450.txt` and `summary_20251027-123450.txt`.
- Script log: `packet_inspection_20251027-123450.log` (also mirrored in Listing 2).

3.4 Observations and Reflections

The capture shows the expected DNS lookups and HTTPS/TLS flows to `example.com`, GitHub endpoints used by VS Code, and ICMP echo traffic aimed at `8.8.8.8`. Packet headers matched

expectations (no rogue ports, consistent TTLs). Background traffic included mDNS and ARP broadcasts from the local LAN, but nothing anomalous.

If an intrusion were suspected, I would replay the pcap in Wireshark, filter by suspicious IPs or TCP flags, follow suspect streams, and correlate timestamps with system logs. Endpoint statistics and protocol hierarchies (Figure 2) help surface unexpected talkers, while display filters (e.g., `tcp.flags.syn == 1 && tcp.flags.ack == 0`) quickly identify scans. Exporting malicious conversations for signature development or sharing with incident responders would be the next step.

4 Overall Reflections and Next Steps

- Part 1 confirmed that minimal, service-specific UFW rules harden the Pi without disrupting Lab 2 functionality.
- Part 2 demonstrated an end-to-end workflow for capturing, archiving, and summarising wireless traffic; the automation scripts make reruns trivial.
- Planned follow-ups include shipping UFW and Tshark logs to a central collector, scheduling periodic verification scans (`nmap`) to detect drift, and extending automation to Step 3 once wireless tuning begins.